

Parallel Implementation of R-trees on the GPU

Lijuan Luo*, Martin D.F. Wong†, and Lance Leong‡

* University of Illinois at Urbana-Champaign

NVIDIA Corporation

Email: liluo@nvidia.com

† University of Illinois at Urbana-Champaign

Email: mdfwong@uiuc.edu

‡ NVIDIA Corporation

Email: lleong@nvidia.com

Abstract—R-tree is an important spatial data structure used in EDA as well as other fields. Although there has been a huge literature of parallel R-tree query, as far as we know, our work is the first successful one to parallelize R-tree query on the GPU. We also propose the first R-tree construction method on the GPU. Unlike the other parallel construction methods, our method does not depend on a partition algorithm and guarantees the same quality as the sequential construction. Experiments show that more than 30x speedup on R-tree query and more than 20x speedup on R-tree construction are achieved.

I. INTRODUCTION

The R-tree is an index data structure for the efficient management of spatial data such as rectangles. For example, to manage the shaded rectangles in Fig. 1, we can construct an R-tree as shown in Fig. 2. One of the most common operations on an R-tree is the range query: given a query rectangle, retrieve all the data rectangles that intersect it. A lot of EDA problems can be formulated as range query problems. For example, in design rule checking (DRC), overlap checking is a range query problem; spacing checking can also be solved by range query if we extend the boundary of the checked object by the minimum spacing amount. Another example is the via dropping problem in flip-chip packaging. To obtain all the legitimate via locations for the power nets, we look for the intersections between the VDD/GND wires on the top-metal layer and the VDD/GND wires on the redistribution layer. In extreme ultraviolet lithography (EUVL), we check the overlaps between patterns/layouts and the defects. The key idea is to assure the patterns/layouts on the mask blank where the defects do not impact the final wafer image [1]. To efficiently solve these query problems, we use the R-tree structure.

There have been a lot of parallel works on R-tree query on different multiple-processor architectures [2], [3], [4], [5]. All of them tried to maximize the parallelism for large range queries on the particular architectures. Some of them also managed to improve the throughput by engaging as few disk I/Os as possible. In this paper, we assume the spatial data can fit into memory, which is reasonable given that memory is getting larger and cheaper. Also we want to improve parallel R-tree query on the GPU. As far as we know, there is only one existing R-tree work on the GPU [6]. However, no speedup result is shown in [6]. Moreover, the work in [6] is not realistic

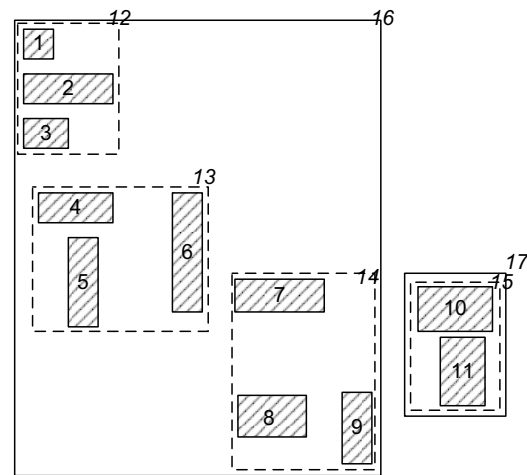


Fig. 1. Data (shaded rectangles) organized in a lowx packed R-tree.

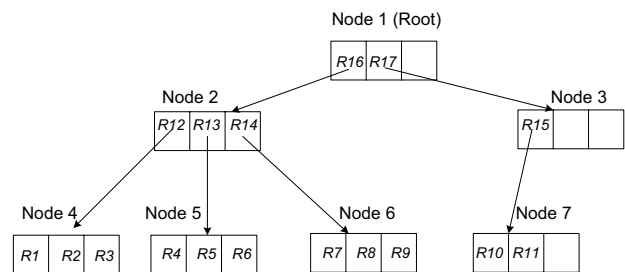


Fig. 2. R-tree structure corresponding to Fig. 1.

by assuming the R-tree can be completely loaded into the GPU shared memory, which is only 16 KB. Later on, we will also show that the implementation introduced in [6] does not fully explore the parallelism in R-tree query. Hence, our work is the first one which successfully uses the GPU to improve the speed of R-tree query.

There are typically two ways to construct an R-Tree. The first one is to repeatedly insert each data item. This method is often used when the database is frequently changing. Sometimes the database is static, such as in the above-mentioned applications in EDA. In this case, it is preferable to insert all of the data items using a single operation, or bulk loading [7]. In this paper, we will also present a parallel bulk loading

algorithm on the GPU. In the following discussion, we use R-tree construction and bulk loading interchangeably. Previous works on parallel R-tree construction [2], [8], [9] choose to partition the data set into several groups, build small R-trees in parallel and then combine the small R-trees into the final R-tree. They have faced the problems of choosing an appropriate partition algorithm, balancing the heights of individual R-trees, optimizing the data transfer among different processors and so on. In our implementation, we take a completely different way to parallelize the R-tree construction. We only build a single R-tree and parallelize the two basic operations in R-tree construction: sorting and packing. This method is a better fit for the massive parallel architecture of the GPU. Also without partitioning the data to individual R-trees, we can guarantee the same quality of R-tree construction as the sequential implementation.

II. BACKGROUND

R-tree is one of the most promising spatial data structures [10]. A geometric object is represented by its minimum boundingbox rectangle or MBR. Each leaf node of an R-tree holds two items for each data record. One is the MBR of the record, and one is a pointer (or an identifier) to the data record itself. In the following discussion, we will assume each data object is simply a rectangle. Therefore, we use the terms *MBR of the data record* and *data rectangle* interchangeably. Similarly, each non-leaf node of an R-tree holds two items for each of its children: the MBR that covers all the rectangles in the child, and a pointer to the child. The main innovation in the R-tree is that the nodes (to be precise, the MBRs of nodes) are allowed to overlap. This way, the R-tree can guarantee at least 50% space utilization and remain balanced [11]. Fig. 1 illustrates data rectangles (shaded rectangles) organized in an R-tree with fanout 3. The *fanout* of an R-tree is defined as the maximum number of children of each non-leaf node or the maximum number of data records in each leaf node.

The NVIDIA GeForce GTX 480 graphics processor is a collection of 15 multiprocessors, with 32 streaming processors each. The 15 multiprocessors share one off-chip global memory. Note that global memory is not cached, so it is very important to achieve *memory coalescing*. Memory coalescing happens when consecutive threads access consecutive memory locations. In this case, several memory transactions can be *coalesced* into one transaction. The 15 multiprocessors also share a small read-only constant memory with cache. Within each multiprocessor there is a shared memory, which is common to all 32 streaming processors inside the multiprocessor. The shared memory is on-chip memory and is only 16 KB.

We use the NVIDIA CUDA computing model to do parallel programming on the GPU. A typical CUDA program consists of several phases that are executed either on the host CPU or on the GPU. The CPU code does the sequential part of the program. The phases that exhibit rich parallelism are usually implemented in the GPU code, called *kernels*. The action of calling a GPU function from the CPU code is termed *kernel launch*. When a kernel is launched, a two-level thread

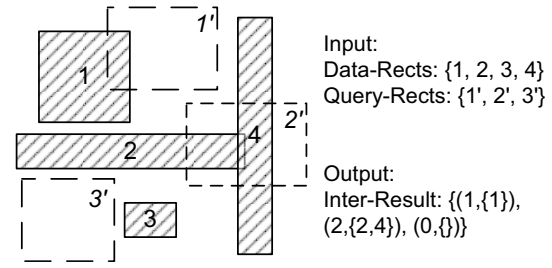


Fig. 3. An example of R-tree query problem.

structure is generated. The top level is called *grid*, which consists of one or more *blocks*. Each block consists of the same number of threads. The whole block will be assigned to one multiprocessor.

III. PARALLEL R-TREE QUERY

In this section, we discuss the GPU implementation of R-tree query. Note that our R-tree query method is independent of the R-tree construction method.

A. R-tree Data Structures on the GPU

Given a list of data rectangles *Data-Rects* and a list of query rectangles *Query-Rects*, the R-tree query problem is to find a list of intersection results *Inter-Result*. The *i*th item in the resulting list has the form of:

$$(\#Intersection, list_of_data_rectangles)$$

Here the first element $\#Intersection$ is the number of data rectangles that intersect with the *i*th query rectangle. The second element is the list of identifiers of the intersecting data rectangles. Fig. 3 shows an example R-tree problem. Here, all the data rectangles are represented by shaded rectangles with solid boundaries and all the query rectangles are represented by clear rectangles with dashed boundaries.

We use two arrays to store the R-tree structure in the GPU global memory. The first is an array of node structures called *Index*. The second is an array of rectangle coordinates, called *Rect*. The size of each array is equal to the number of R-tree nodes multiplied by the fanout of the R-tree. Take Fig. 2 for example; we have 7 R-tree nodes and the fanout is 3, so the sizes of the two arrays are both equal to 21; each R-tree node is corresponding to 3 consecutive array items. Fig. 4 shows part of the arrays corresponding to the R-tree in Fig. 2. The numbers on the top of Fig. 4(a) and (b) are the array indices.

For an arbitrary array item $Index[(i-1) \times 3 + (j-1)]$ with $j = 1, 2, 3$ and $i = 1, 2, \dots$, if Node *i* is a non-leaf node, then this array item represents the starting index of the *j*th child in the *Index* array; if Node *i* is a leaf-node, then this array item is the opposite number of the identifier of the node's *j*th data rectangle. For example in Fig. 4, $Index[0]$ is equal to the starting index of the first child of Node 1. $Index[12]$ is equal to -4, because the 1st rectangle of Node 5 is the 4th input data rectangle. We use 0 to represent both an empty child and a non-existing data rectangle. Each array item $Rect[(i-1) \times 3 + (j-1)]$ where $j = 1, 2, 3$ and $i = 1, 2, \dots$

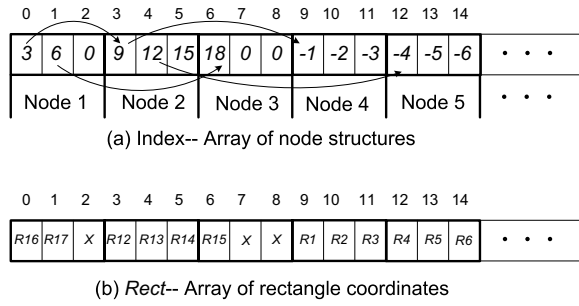


Fig. 4. The GPU data structure of the R-tree in Fig. 2.

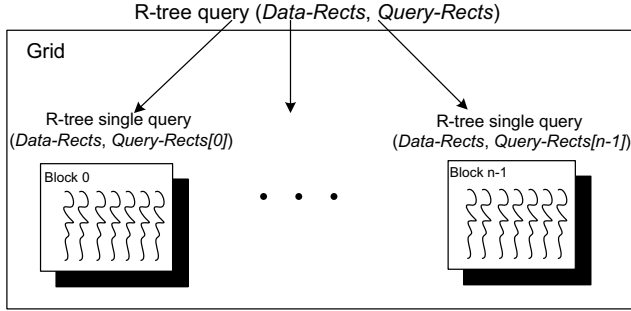


Fig. 5. The two-level parallelism idea for R-tree query.

is a vector, representing the coordinates of the left-bottom and the right-top corners of the j th rectangle in Node i . We use a cross in Fig. 4(b) to represent *don't care*. We know the rectangle item does not exist because the corresponding item in the *Index* array is equal to 0.

B. Parallelism Ideas

The R-tree query problem shows a great amount of parallelism and is a natural fit for the GPU architecture. The first level of parallelism is obvious. Assume the size of *Query-Rects* is n ; we can easily achieve n -way parallelism by assigning one thread for each query rectangle. However, a single query on an R-tree (we will refer to this problem as *R-tree single query* problem) is too complicated for the light-weighted GPU thread. Hence, we need to explore another level of parallelism, using multiple threads to perform each query operation. Our overall idea is to follow the two-level thread hierarchy of the GPU. We launch n blocks to handle the R-tree query problem with n query rectangles. Then each block of threads is dedicated to solving one R-tree single query problem in parallel. Fig. 5 is the pictorial explanation of our two-level parallelism idea where each curve represents a thread.

The R-tree single query problem is very similar to the breadth-first search (BFS) problem in [12]. We start with the root node and push it into a frontier queue. If the MBR of the root node does not intersect the query rectangle, we can terminate the search. Otherwise, we pop out the root node and push all the child nodes of the root node into the queue and go to the next level. In the next level, we pop out each frontier node (a frontier node is defined as a node which is going to be checked in current level) and check its MBR against

the query rectangle. Similarly if a frontier node intersects the query rectangle, we push the child nodes of the frontier node into the queue. This process continues until the queue becomes empty.

R-tree single query problem is different from the BFS problem in that we are given a tree structure as input instead of a graph structure. Also this tree structure is at least 50% full. Assume the fanout of the R-tree is d ; 50% full means that averagely each node has at least $\frac{1}{2}d$ non-empty entries. We can take advantage of these properties and explore more parallelism. The GPU implementation of [12] is node-based parallelism, meaning that each thread is dedicated to one frontier node. Here to solve the R-tree single query problem, we employ entry-based parallelism: each thread is dedicated to one entry (including an empty entry) of a frontier node. Hence if the frontier has m nodes, we will have $d \times m$ threads and at least half of them are doing meaningful work. Since consecutive threads are corresponding to consecutive entries in the *Index* and *Rect* arrays, our entry-based method can guarantee memory coalescing.

Our implementation works better than the R-tree single query of [6] in two aspects. First, the implementation in [6] assumes the whole R-tree can fit into shared memory, which is not realistic since the shared memory is only 16 KB in current GPU device. Second, node-based parallelism is used in [6], which explores less parallelism than our entry-based method.

We can further improve the performance by using the GPU constant memory to store part of the R-tree. Since all the threads are accessing the same R-tree from top to bottom, we choose to store the top levels of the R-tree in the constant memory to maximize the cache hits. Experimental results show that by employing constant memory, we can reduce the run time by around 10%.

The above implementation works best when each query rectangle has a great amount of overlap with the data rectangles. However, many EDA problems, such as the DRC problem, involve very few overlaps. In this case, the frontier within each block is typically very small and we will not have enough threads to fully use all the computing resources or to hide the latency of memory access. To handle the sparse overlap problems, we propose a method to handle multiple query rectangles in one block by encoding the query index into the frontier node. Fig. 6 demonstrates this idea. Here we handle two query rectangles in one block. We still only maintain one queue structure. But each frontier node is composed of two parts. The highest bit indicates the corresponding query rectangle (Fig. 6 (b)) and the other bits indicate the R-tree node. The reason we want to encode the frontier node instead of maintaining a separate queue is that the queue operation on the GPU is very expensive [12] but arithmetic operations are cheap considering the high GFLOPS current GPUs can achieve.

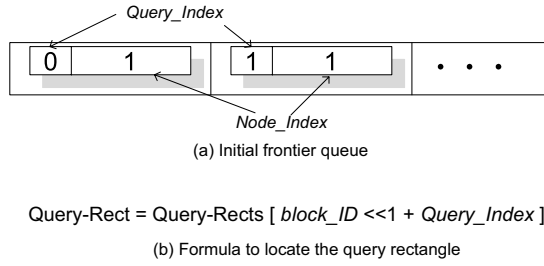


Fig. 6. Frontier queue structure handling two R-tree query requests in one block.

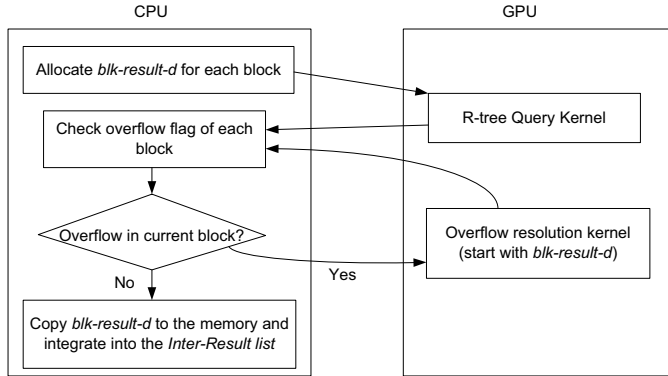


Fig. 7. The flow of R-tree query. The GPU global memory array *blk-result-d* stores the data rectangles that intersect the query rectangle(s) when no overflow happens or the frontier queue right before the overflow happens.

C. Handling Overflow Cases

The amount of overlap between each query rectangle and the data rectangles can vary a lot. In this section, we will discuss how to handle the imbalance problem on the GPU. For the convenience of explanation, we assume each block only handles a single query request.

Before we launch a kernel for R-tree query, we need to allocate the GPU global memory space to hold the R-tree query results and the shared memory space to hold the frontier queues. We can estimate the memory space based on the property of the specific application or by sampling several query rectangles and evaluating the average amount of overlap.

In each block we do the R-tree query level by level. Therefore, if overflow happens, we can always go back to the previous level and store the status of the level right before the overflow. Then we switch on the overflow flag for the current block. Once the R-tree query kernel is terminated, we can check the overflow flag of each block and handle the overflow case if there is any. We launch a new kernel to handle each overflow case. In this new kernel, we will start with the level right before the overflow and continue the remaining levels of query in parallel. Note that in the overflow resolution kernel, multiple blocks are dedicated to checking and storing the overlaps originally assigned to one block. Therefore, the chance of another overflow is greatly reduced. Even if there is still overflow, we can launch the overflow resolution kernel again, and eventually the overflow resolution can be guaranteed. Fig. 7 shows the overall flow of R-tree query.

```

1: Let level  $l$  equal to 0
2: repeat
3:   if  $l = 0$  then
4:     Sort data rectangles on ascending x-coordinate
       of the left-bottom corner
5:     while more rectangles do
6:       generate a new R-tree node
7:       assign the next  $d$  rectangles to this node.
8:     end while
9:   else
10:    while more nodes in level  $l - 1$  do
11:      generate a new R-tree node
12:      add the next  $d$  nodes of level  $l - 1$  as a child
        node of this new R-tree node
13:    end while
14:    if there is only one node in this level then
15:      mark this node as the root node
16:    end if
17:  end if
18: until get the root node

```

Fig. 8. Pseudo-code of the bulk loading algorithm.

IV. PARALLEL R-TREE CONSTRUCTION

There are two approaches to bulk loading, the bottom-up approach such as [13] and [11], and the top-down approach such as [14] and [7]. Since both approaches are composed of the same basic steps, sorting and packing, these approaches are not fundamentally different when it comes to parallel implementation. In our work, we implement a parallel version of bottom-up R-tree construction. The parallelism ideas can be migrated to the top-down approaches.

Roussopoulos and Leifker [13] proposed the first method of bottom-up bulk loading. The idea is to sort the data on the x coordinate of the left bottom corners of the rectangles. (Hence the method is called *lowx pack*.) The sorted list of rectangles is scanned; successive rectangles are assigned to the same R-tree leaf node, until this node is full; then, a new leaf node is created and the scanning of the sorted list continues. The same process is applied to the MBRs of the leaf nodes to build another level of the R-tree. This process is applied iteratively until the root node of the R-tree is obtained. Fig. 1 shows how the data rectangles are packed using this approach. Fig. 8 shows the pseudo code of this algorithm. Here we assume the fanout of the R-tree is d .

The bulk loading algorithm is composed of two building blocks. The first one is sorting (Step 4 in Fig. 8). The second one is the while loops (Step 5-8 and Step 10-13), or packing. In the following, we will discuss how to implement these two basic components on the GPU.

We use radix sort as the sorting engine on the GPU because it is a natural fit for parallelism and is among one of the most efficient sorting algorithms. We employ the radix sort kernel provided by CUDA SDK [15]. Note that in the *lowx pack* algorithm we only need to sort the rectangles once at the leaf level. The MBRs in the higher levels are automatically sorted.

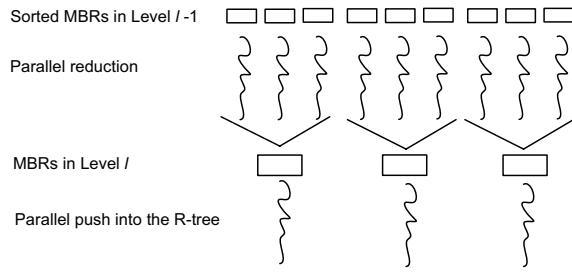


Fig. 9. Parallel R-tree packing.

TABLE I
BENCHMARK PROPERTY

	#Data-Rects	#Query-Rects	Average Overlap	Maximum Overlap
Ex1	30,674	30,674	3	15
Ex2	76,999	76,999	3	17
Ex3	402,467	402,467	3	21
Ex4	2,249,727	2,249,727	5	358

If other sorting criteria are used, we may need to call the sorting procedure for every level.

Packing is another highly parallel operation because all the new R-tree nodes of the same level can be built in parallel. Most of the computation for packing comes from calculating the MBRs of the new R-tree nodes. As shown in Fig. 9, the MBR of the new R-tree node can be obtained by the reduction operations (max and min) on the MBRs of its child nodes.

Note that Kamel proposed a better way to sort the data based on the Hilbert curve in [11]. Since our parallelization method does not depend on the sorting keys, it can be used to accelerate [11], too.

V. EXPERIMENTAL RESULTS

We conducted experiments on a host machine with 3.33 GHz Intel Xeon processor and 12 MB cache. A single NVIDIA GeForce GTX 480 processor with 1.5 GB global memory was used to run CUDA applications. We downloaded the test cases from [16]. Each test case is a list of rectangles. We use them as both data rectangles and query rectangles. Table I shows the properties of these test cases. Column 1 is the number of data rectangles and Column 2 is the number of query rectangles, which are the same in our test. Column 3 is the average number of overlaps between each query rectangle and all the data rectangles. Column 4 is the maximum number of overlaps between each query rectangle and all the data rectangles. We downloaded the R-tree sequential implementation from [17] as the baseline implementation.

To test the speedup of R-tree query on the GPU, we copy the R-tree generated by the sequential code [17] from the CPU to the GPU and only compare the R-tree query times. Table II shows the speedup results.

The GPU time reported in Table II includes the time to copy the query results back from the GPU memory to the CPU memory as well as the extra time to handle overflow cases. More speedup is achieved as the problem size grows. For the biggest benchmark, we can achieve almost 35 times speedup.

TABLE II
R-TREE QUERY TIME COMPARISON

	CPU Query Time (ms)	GPU Query Time (ms)	Speedup
Ex1	62.90	2.79	22.54
Ex2	180.03	6.36	28.31
Ex3	1107.81	36.62	30.25
Ex4	11594.00	331.52	34.97

TABLE III
R-TREE CONSTRUCTION TIME COMPARISON

	CPU Construction (ms)	GPU Construction (ms)	Speedup
Ex1	12.56	3.71	3.39
Ex2	31.90	5.65	5.64
Ex3	179.23	13.62	13.16
Ex3	1037.28	45.08	23.01

The next experiment is to test the speedup we can achieve in bulk loading. Note that the R-tree construction in [17] is done by repeatedly inserting nodes. For fair comparison, we modify the code in [17] and use the same lowx pack technique on the CPU. The CPU code of radix sort is obtained from [18]. Table III shows the run time comparison.

VI. APPLICATION TO VIA DROPPING

The R-tree query technique can be used to accelerate the via dropping process in flip-chip packaging. Flip chip is a method for interconnecting semiconductor devices to external circuitry with solder bumps. Fig. 10 is the pictorial description of flip-chip packaging. In the packaging process, we introduce an extra routing layer above the top-metal layer, which is called redistribution layer or RDL (Fig. 11). RDL routing is to connect solder bumps to the pad openings of the die. A huge amount of routing on RDL as well as on the top metal is PG (power/ground) routing with the task of supplying power all over the die. In order to conduct more current, we want to drop as many vias as possible to connect the RDL and top metal routing. Apparently we can obtain all the legitimate via areas by computing the overlaps between the RDL wires and the top-metal wires of the PG net. According to the data we get from industry, there can be thousands of PG wires on the top metal and tens of thousands of PG wires on RDL which can hold up to 100K vias or sometimes even millions of vias. To handle overlap calculation of this scale, R-tree is a preferred data structure.



Fig. 10. The flip-chip package.



Fig. 11. The redistribution layer and top-metal layer.

TABLE IV
PROPERTIES OF TEST CHIPS

test chip	signal	#rdl-wires	#top-metal-wires
chip1	vdd	13,538	4,628
chip1	gnd	12,308	1,895
chip2	vdd	87,244	5,725
chip2	gnd	82,303	5,794

TABLE V
TIME TO CALCULATE OVERLAPS BETWEEN RDL AND TOP-METAL

Case	#Data-Rects	#Query-Rects	Avg/Max Overlap	CPU (ms)	GPU (ms)	Speedup
chip1(vdd)	13,538	4,628	7/218	10.93	3.44	3.18
chip1(vdd)	4,628	13,538	2/39	15.68	1.53	10.24
chip1(gnd)	12,308	1,895	17/206	6.47	1.98	5.83
chip1(gnd)	1,895	12,308	2/23	15.09	1.54	9.80
chip2(vdd)	87,224	5,725	33/288	44.52	7.19	6.06
chip2(vdd)	5,725	87,224	2/23	108.13	7.19	15.04
chip2(gnd)	82,303	5,794	33/295	42.17	7.32	5.76
chip2(gnd)	5,794	82,303	2/18	132.20	7.99	16.55

We can use RDL wires as the data rectangles and top-metal wires as the query rectangles or the other way around. In our test cases, there are usually more RDL wires than top-metal wires, so the first option will take more time to build the R-tree but less time to finish the actual overlap query. Also if we need to adjust the routing in one layer frequently, it is better to use the wires in the other layer as data rectangles. We extracted the RDL wires and top-metal wires from two real chip designs. The properties of these two test chips are summarized in Table IV.

Table V shows that up to 16x speedup can be achieved for R-tree query. For the same net of the same test chip, more speedup can be achieved when using the top-metal wires as data rectangles. This is mainly due to two reasons. First, more query rectangles (there are usually more RDL wires than top-metal wires) mean more parallelism on the GPU, hence more speedup. Second, using RDL wires as query rectangles creates a more uniform overlap situation (the gap between the maximum overlap and the average overlap is smaller), which favors the parallel architecture.

Table VI shows the time of R-tree construction on both the CPU and the GPU. Up to 6x speedup can be achieved when using bulk-loading to create R-trees. However, for small test cases, bulk loading on the GPU can be even slower than on the CPU. But the construction time for these cases is much

TABLE VI
TIME TO CONSTRUCT R-TREES FOR EITHER THE RDL WIRES OR THE TOP-METAL WIRES.

Case	#Data-Rects	CPU (ms)	GPU(ms)	speedup
chip1(vdd)	13,538	6.05	3.76	1.61
chip1(vdd)	4,628	2.12	2.96	0.72
chip1(gnd)	12,308	5.48	3.47	1.58
chip1(gnd)	1,895	0.88	3.41	0.26
chip2(vdd)	87,224	39.44	6.24	6.32
chip2(vdd)	5,725	2.53	3.04	0.83
chip2(gnd)	82,303	36.23	5.96	6.08
chip2(gnd)	5,794	2.52	3.04	0.83

less than the query time and is not the bottleneck of the via dropping problem.

VII. CONCLUSION

We have presented effective GPU implementations of R-tree query and R-tree construction. The proposed R-tree query method is general and robust. It is able to handle both dense and sparse overlap situations as well as overflow cases. We have also shown how the GPU-based R-tree structure is used for efficient via dropping in flip-chip packaging.

REFERENCES

- [1] J. Burns and M. Abbas, "EUV mask defect mitigation through pattern placement," in *Proc. SPIE, Vol. 7823, Poster Session*, Monterey, CA, Sep. 2010, p. 782340.
- [2] B. Schnitzer and S. T. Leutenegger, "Master-client R-trees: A new parallel R-tree architecture," in *Proc. International Conference on Scientific and Statistical Database Management*, Cleveland, OH, July 1999, pp. 68–77.
- [3] B. Wang, H. Horinokuchi, K. Kaneko, and A. Makinouchi, "Parallel R-tree search algorithm on DSVM," in *Proc. International Conference on Database Systems for Advanced Applications*, Hsinchu, Taiwan, 1999, pp. 237–244.
- [4] H. S. Erik and G. Hoel, "Performance of data-parallel spatial operations," in *Proc. International Conference on Very Large Data Bases*, Santiago de Chile, Chile, Sep. 1994, pp. 156–167.
- [5] I. Kamel and C. Faloutsos, "Parallel R-trees," in *Proc. ACM International Conference on Management of Data (SIGMOD'92)*, San Diego, CA, June 1992, pp. 195–204.
- [6] M. Kunjir and A. Manthramurthy, "Using graphics processing in spatial indexing algorithms," 2009. [Online]. Available: http://dsl.serc.iisc.ernet.in/mayuresh/graphics_report.pdf
- [7] H. Alborzi and H. Samet, "Execution time analysis of a top-down R-tree construction algorithm," *Inf. Process. Lett.*, vol. 101, no. 1, pp. 6–12, Jan. 2007.
- [8] A. Papadopoulos and Y. Manolopoulos, "Parallel bulk-loading of spatial data," *Parallel Computing*, vol. 29, no. 10, pp. 1419–1444, Oct. 2003.
- [9] A. Cary, Z. Sun, V. Hristidis, and N. Rish, "Experiences on processing spatial data with MapReduce," in *Proc. of the 21st International Conference on Scientific and Statistical Database Management*, New Orleans, LA, June 2009, pp. 302–319.
- [10] A. Guttman, "R-trees: A dynamic index structure for spatial searching," in *Proc. ACM International Conference on Management of Data (SIGMOD'84)*, Boston, MA, June 1984, pp. 47–57.
- [11] I. Kamel, "On packing R-trees," in *Proc. International Conference on Information and Knowledge Management*, Washington, DC, Nov. 1993, pp. 490–499.
- [12] L. Luo, M. D. F. Wong, and W. Hwu, "An effective GPU implementation of breadth-first search," in *Proc. ACM/IEEE Design Automation Conference (DAC'2010)*, Anaheim, CA, June 2010, pp. 52–55.
- [13] N. Roussopoulos and D. Leifker, "Direct spatial search on pictorial databases using packed R-trees," in *Proc. ACM International Conference on Management of Data (SIGMOD'85)*, Austin, TX, May 1985, pp. 17–31.
- [14] Y. J. Garcia, M. A. Lopez, and S. T. Leutenegger, "A greedy algorithm for bulk loading R-trees," in *Proc. ACM International Symp. on Advances in Geographic Information Systems (GIS'98)*, Washington, DC, Nov. 1993, pp. 490–499.
- [15] "NVIDIA CUDA SDK," 2009. [Online]. Available: <http://www.nvidia.com/cuda>
- [16] Y. Theodoridis, "The R-tree-portal," 2003. [Online]. Available: <http://www.rtreeportal.org>
- [17] [Online]. Available: <http://superliminal.com/sources/sources.htm>
- [18] P. Terdiman, "Radix sort revisited," 2000. [Online]. Available: <http://www.codecorner.com/RadixSortRevisited.htm>

VIII. ACKNOWLEDGEMENT

This work was partially supported by the National Science Foundation under grant CCF-1017516.